Actionability as an Explanation of Grid Cells: A Critical Review and Reproduction of Dorrell et al. (2023)

Tafara Chibanda (2683459)
University of the Witwatersrand

Nathan Le Roux (2717495)
University of the Witwatersrand

Summary

Dorrell et al. (2023, ICLR) ask why the brain would represent two-dimensional position with grid cells. Their answer is that a spatial code should be *actionable*: every movement (a step north, then east, and so on) should change the population activity by the same matrix wherever the animal starts, so that a route never travelled can still be predicted. Representation theory restricts each neuron's tuning to a sum of spatial sinusoids. Learning determines their amplitudes and wave vectors. Gradient descent adjusts these parameters in a 64-neuron population. The objective separates distant locations, subject to non-negative and bounded firing. The optimum, they report, is several modules of roughly hexagonal grid cells. They explain this with analytic and heuristic arguments: non-negativity needs a constant offset or harmonically related frequencies, which gives a lattice of frequencies; the animal's occupancy and the resolution it needs select a ring of useful frequencies that a hexagonal lattice fills most densely; and a small neural length scale σ favours non-harmonic frequencies, hence several modules. Three predictions follow (module size against the squared ratio of grid spacing to field width, small offsets between module orientations, and changes with room shape), compared informally with Stensola et al. (2012). We read the optimality claim as a numerically supported heuristic rather than a proof, since the paper's Appendix G calls the problem non-convex; this is the point in our understanding we are least sure of.

Originality

Earlier accounts work at other levels. Continuous-attractor models (Burak and Fiete, 2009) hand-tune a circuit that integrates velocity; efficient-coding analyses, as the authors note, assume grid-like tuning. Sorscher et al. (2019) obtain hexagonal grids from non-negativity and a place-cell read-out target and, in the authors' view (Appendix H.2), modules only through a quantisation effect. Closest is Gao et al. (2021): they also use a position-independent linear update (a group representation), but add an isotropy term that favours hexagons directly and obtain several modules only by imposing block-diagonal updates. Dorrell et al. keep the linear action, drop the isotropy term, derive hexagons from the band of useful spatial frequencies, and derive modules from σ. The contributions are an interpretable family of codes, modules without imposed block structure, and quantitative predictions.

The weaknesses follow from the framework. Requiring a location-independent *linear* action is a substantive restriction, so periodic tuning is largely a consequence of the formulation, and "grid cells from minimal constraints" means hexagonal and modular codes *within* linear-action models. The claim that isotropy is unnecessary (Appendix H.1) is argued but never tested by running the authors' objective with and without it, and there is no head-to-head comparison with Sorscher et al. (2019).

Level of abstraction. The paper is computational: it has no spiking, learning rule, velocity input or noise. The model is more abstract than attractor or trained-network models. It covers variables whose transformations form a group, including the circle, sphere and 3-D space (Appendix M). Its linear update rule is more restrictive. We agree with this choice for a "why" question, because it lets the authors derive rather than fit. The cost is that the link to attractor circuits, which they leave as future work, stays open.

Clarity

The paper is well organised. Section 2 introduces a circle before the plane. Figure 4A explains the frequency-ring argument, and Figure 6 shows every neuron. The authors acknowledge simplifications ("this is a bad loss", Section 3.1). Our difficulties were specific:

1. Table B.3 gives the settings of Figure 2B only. Figures 8 and 9 change σ, l and the number of steps and rescale the initial constraint weights with the loss size, without giving values.
2. The number of frequencies D and the initial variances ("as in Table B.3") are not in the table.
3. The norm target $k_n = 4$ is a log-target (equation 26), but the equation 25 penalty is an average squared error that is small when the constraint holds. A target of $e^4 \approx 55$ exceeded the measured norm penalties and, in our runs, left the constraint inactive (see Reproducibility).
4. Equation 22 normalises by a *sum* over M sampled points while the main text uses expectations, so the numerical meaning of σ depends on M.
5. Figure 9 counts "modules" and Figure 8 judges "hexagonal" without saying how, and the Figure 2B caption omits σ, N and l.
6. Figure 5 holds nine panels, and its comparisons with data (5B, 5E, 5I) are by eye, without an effect size.

Inspection of the public code (commit 940659e, authors' repository) found that its norm routine aggregates unnor-

malised responses where equation 25 uses normalised ones, and that its `sigma_sq` argument must be distinguished from the paper's σ. These differences motivate per-figure configuration files. They do not establish errors in the reported results.

Quality

Strengths. Each ingredient is tied to a mechanism that makes a checkable statement: non-negativity forces harmonics and a constant offset, the two spatial scales select a ring of frequencies, and σ sets how many modules arise. The last held in our runs: at $l = 0.5$ the number of modules rose from 1.0 at $\sigma = 1.0$ to 9.5 at $\sigma = 0.1$ (Table A1). The authors also volunteer limits (Appendix B.4.2: "the more modules the less hexagonal"; their data comparisons are "rudimentary").

Weaknesses. (1) Optimality is heuristic: Appendix G calls the problem non-convex, and the evidence is gradient descent from random starts, which varies with the seed. At $\sigma = 0.2$, nine 50k-step runs gave 4.7$\pm$1.5 modules with 0.3 hexagonal per run on average; three runs at the paper's 150k steps gave 3, 3 and 6 modules, none hexagonal. We did not reproduce Figure 2B under our documented conventions; this does not establish how representative the authors' result was. Even so, "several hexagonal modules" is the weakest part of the central result: the authors concede that more modules are less hexagonal (quoted above), and the main text shows a single clean population. (2) "Minimal" rests on ablations on a small torus (Appendix I). On the plane, removing non-negativity or the functional term destroyed grid structure in our runs, but removing actionability did not (38% of cells still scored above 0.3), so a grid score alone cannot separate lattices from bumpy patterns. (3) The hexagonal outcome appears to depend on how the bounded-activity constraint is enforced (Reproducibility). Appendix B.4 says dependence on optimiser settings "does not seem deeply worrying"; this one changes the result. (4) The predictions are weakly constrained. The scaling $N \propto (\nu/\mu)^2$ is supported in sign but not decisively (slope 1.47, 95% bootstrap interval -0.69 to 3.66, 30 modules), and the $\sim 4^\circ$ offset depends on μ and ν values that are not reported: only 32% of the values we scanned give 3–8°.

Abstraction. We agree with the level for the question "what makes a code useful", less so for the biological reading. The optimiser receives explicit positions and a global loss. Non-negativity and boundedness are penalties. The framework does not specify a biological circuit implementing $T(\Delta x)$. An optimal code is not shown to be learnable, or robust to noise.

Significance

Influence. The paper gives a template: choose a variable, write down the group of its transformations, add biological and functional constraints, and read off the code. The authors already apply it to circles, spheres and 3-D space; for 3-D they suggest that irregular fields in rats and bats mean that animals give up actionability there, an intriguing claim resting on a thin argument (Appendix M). For machine learning, the principle that a representation should let a system predict the consequences of its actions is broadly useful. The theory complements circuit models: it says which codes are useful, and mechanistic models ask whether plausible networks can implement them. Recordings showing toroidal population structure in grid modules (Gardner et al., 2022) make that relationship timely.

Theory–experiment loop. The theory was motivated by known grid properties (hexagons, modules) and returns predictions, but these need sharper tests than visual comparison with Stensola et al. (2012). Three experiments follow directly. (1) *Room geometry.* Figure 5H–I already compares square-to-rectangle recordings from Stensola et al. (2012). A quantitative follow-up should record the same module across these rooms, fit any free parameters in the first room only, and predict which cells keep their scale and which shift. Separate physical walls from sampled occupancy, because the theory enters through $p(x)$. (2) *Module offsets.* Measure μ and ν for each module pair, compute the predicted offset (in our re-implementation it depends strongly on both), and compare with a null of random orientations, whose mean offset is 15°. (3) *Actionability itself.* The core assumption, one position-independent linear map per displacement, has not been tested directly. With simultaneously recorded neurons of one module, fit a map per displacement on part of the arena and predict held-out population vectors; compare its held-out performance with position-dependent and nonlinear alternatives. Success alone would not exclude those alternatives. Failures would feed back into the model, by adding sensory input or noise, as the authors suggest, or the isotropy term of Gao et al. (2021), which the authors say (Appendix H.1) may be needed to make many modules hexagonal. Our runs, where multi-module codes were rarely hexagonal, support trying it. Because several models produce grids, experiments should target where their predictions differ, not whether grids appear.

Reproducibility

We built two implementations. Nathan's (NumPy, hand-derived gradients checked against finite differences, 60 training runs) was written from the paper's text only; the authors' code was deliberately not read. Tafara's (JAX, automatic differentiation, two reference runs and nine controlled 150 000-step runs) inspected the authors' public code (commit 940659e) when choosing initialisation and optimiser conventions, so the two used different information. On identical float64 inputs their loss terms

agree to 2.8×10^{-13} and, after aligning the functional weight, their directional derivatives to 5.1×10^{-11}: both optimise the same equations. Table A1 lists every claim we tested.

What held. A single hexagonal module at large σ (both 150k-step runs at $\sigma = 1.0$, and $\sigma = 0.7$), and a module count that grows as σ falls (Figure A2).

What did not. At $\sigma = 0.2$, three 150k-step runs produced 12 modules, none hexagonal (Figure A3). At Figure 8's $\sigma = 0.4$, $l = 0.5$, Tafara's three seeds gave elongated periodic cells; Nathan's two 50k-step seeds gave opposite hexagonality outcomes. At $l = 0.1$, Tafara's non-hexagonal $N = 64$ result agrees with the paper's band-like/square regime; Nathan's single $N = 100$ run did not recover its hexagonal example. Our module-offset and 1-D harmonic-lattice findings were also incomplete (Table A1). These differences do not establish the frequency of success under the authors' conventions.

Constraints and measurement. The implementations differ in functional weight, initialisation, norm target and Adam convention, so they cannot be ranked as matched training runs. With the literal target e^4, Nathan's norm multiplier reached its floor in all five main runs. Tight targets near 0.02 strengthened the penalty but did not guarantee satisfaction: all three JAX baseline runs exceeded 0.02 on held-out samples. Nathan's single matched $\sigma = 1$ comparison lost its hexagonal module under the tighter target (Table A2). Constraint sensitivity is therefore a hypothesis supported by limited runs, not an established explanation of failed training.

Re-evaluating five Nathan checkpoints with Tafara's fixed window and held-out samples gave norm errors of 1.85–5.83 (Table A3). Broad hexagonal examples received negative median window-based grid scores: the window contains too few repeats to assess their geometry reliably. Nathan's analysis sizes the autocorrelation window to each module's grid period; Tafara's uses one fixed window for every cell. The two scores answer different measurement questions. Negative scores here do not negate the geometric examples.

Finally, a constructed non-negative hexagonal code with nearly invariant exact Gaussian room energy had mean sampled norm penalties 0.0961, 0.0274 and 0.00644 at 50, 150 and 600 points over 128 batches (Figure A6). Valid geometry can therefore miss our 0.02 target through sampling variation. This untrained control motivates calibration; it does not prove why optimisation failed. Together, these checks support the paper's geometric possibility while identifying numerical conventions, constraint calibration and measurement scale as requirements for convincing reproduction.

Limits. We ran one to nine seeds per setting, our thresholds for "module" and "hexagonal" are not the paper's, and grid scores are descriptive. We did not attempt room geometry, 3-D, or the paper's torus ablations. These results identify choices needed for reproduction. They do not refute the paper.

References

Burak, Y., & Fiete, I. R. (2009). Accurate path integration in continuous attractor network models of grid cells. *PLoS Computational Biology, 5*(2), e1000291. https://doi.org/10.1371/journal.pcbi.1000291

Gao, R., Xie, J., Wei, X.-X., Zhu, S.-C., & Wu, Y. N. (2021). On path integration of grid cells: Group representation and isotropic scaling. *Advances in Neural Information Processing Systems, 34*. https://arxiv.org/abs/2006.10259

Gardner, R. J., Hermansen, E., Pachitariu, M., Burak, Y., Baas, N. A., Dunn, B. A., Moser, M.-B., & Moser, E. I. (2022). Toroidal topology of population activity in grid cells. *Nature, 602*, 123–128. https://doi.org/10.1038/s41586-021-04268-7

Sorscher, B., Mel, G. C., Ganguli, S., & Ocko, S. A. (2019). A unified theory for the origin of grid cells through the lens of pattern formation. *Advances in Neural Information Processing Systems, 32*.

Stensola, H., Stensola, T., Solstad, T., Frøland, K., Moser, M.-B., & Moser, E. I. (2012). The entorhinal grid map is discretized. *Nature, 492*(7427), 72–78.

Author contributions

Nathan Le Roux ran the NumPy implementation and its 60-run campaign. Tafara Chibanda ran the JAX implementation, the controlled diagnostics, the constructed control and the cross-implementation equation check, and prepared the first review draft. Both authors reviewed and agreed with the review's judgements and confirmed their contributions.

Appendix: tables

Table A1: Every claim of the paper that we tested, with what we found (NumPy: Nathan's implementation, 60 runs, 64 neurons, $L = 1$, steps as stated; JAX: Tafara's implementation, 150 000 steps). Section and figure numbers refer to the paper. "Hexagonal" is judged from the autocorrelogram of the whole plane (six nearest peaks within 15° of a hexagon, lengths within 25%, gridness above 0.3), thresholds that are ours; JAX grid scores are descriptive.

Claim (paper)	What we found	Verdict
One hexagonal module at large σ (Figs. 4B, 8, 9)	NumPy, 150k steps, two runs at $\sigma = 1.0$: modules 1 and 1, hexagonal modules 1 and 1; also $\sigma = 0.7$ (and $l = 0.2$)	Held
More modules as σ falls (Sec. 3.3, Fig. 9)	NumPy, $l = 0.5$, 50k steps: 1.0 modules at $\sigma = 1.0$ rising to 9.5 at $\sigma = 0.1$	Held
Several hexagonal modules at $\sigma = 0.2$ (Fig. 2B)	NumPy, 150k steps, 3 seeds: 3/3/6 modules, 0 hexagonal, at most 14% of cells above 0.3. 50k steps, 9 seeds: 4.7 modules, 0.3 hexagonal per run	Not reproduced
One hexagonal module at $\sigma = 0.4$, $l = 0.5$, $N = 64$ (Fig. 8)	JAX, 150k steps: median grid score 0.050 (seed 0), 0.048–0.065 over 3 seeds, elongated periodic cells. NumPy, 50k steps: hexagonal in one of two seeds (1 and 2 modules)	Not reproduced (mixed)
$N = 100$ gives a hexagonal module at $l = 0.1$ (Fig. 8)	NumPy, 50k steps, one run: 3 modules, 0 hexagonal ($N = 64$: 3 modules, 0 hexagonal)	Not reproduced (one run)
Non-negativity gives a harmonic lattice (Sec. 3.1, Fig. 3C)	NumPy, 1-D line, simplified loss: mean/RMS 0.66 (single sinusoid: 0.82), 6.8% of values negative; 29% of power within 0.1 of an integer multiple (random: 20%)	Partly
All three ingredients needed (App. I, Fig. 15)	NumPy, 50k steps, one run each: without non-negativity 20 modules, none hexagonal, 2% of cells above 0.3; without the functional term 1 module, 0%; without actionability ($D = 96$) 6 modules, 2 hexagonal, 38% of cells above 0.3	2 of 3
$N \propto (\nu/\mu)^2$ (Sec. 4.1)	NumPy: log–log slope 1.47 (95% bootstrap interval -0.69 to 3.66; paper: 2), Spearman 0.59, 30 hexagonal modules from 25 runs	Consistent, not decisive
Neighbouring modules differ by $\sim 4^\circ$ (Sec. 4.2, App. K)	NumPy re-implementation of App. K gives 4.0° only for $(\mu, \nu) = (0.030, 0.14)$, values the paper does not give; 32% of scanned values fall in 3–8°. Measured: 6 pairs, none in 3–8°, median 5.8° (null 15°)	Not reproduced
Table B.3 suffices for Fig. 2B	D and initial variances missing. Literal $k_n = 4$: multiplier at its floor in 8 of 8 50k-step runs and in all five main runs; norm loss 1.1–5.7 (main)	Not sufficient
Same equations in both implementations	Loss terms agree to 2.8×10^{-13}, directional derivatives to 5.1×10^{-11} (float64, shared fixture, NumPy core at commit 7db3147; check passes)	Agree
Not attempted	Room-geometry predictions (Sec. 4.3, Fig. 5F–I), the 3-D case (Fig. 19), the torus ablations as drawn in Fig. 15	—

Table A2: The norm target in both implementations. Targets are on the loss of equation 25: the literal target is e^{k_n} with $k_n = 4$, the tight ones are $e^{-4} \approx 0.018$ (NumPy) and 0.02 (JAX). NumPy rows with 50k steps compare a snapshot of the 150k-step literal run, or the eight additional literal baseline runs, with single tight runs; the JAX rows use three matched seeds. λ_n is the final norm multiplier.

Setting	Norm target	Final norm loss	Outcome
NumPy, $\sigma = 1.0$, 50k steps, seed 0	literal e^4	inactive (λ_n at floor at 150k)	1 module, 1 hexagonal, 100% of cells above 0.3
	tight e^{-4}	0.021 ($\lambda_n = 2.0$)	1 module, 0 hexagonal, 0% above 0.3; banded cells
NumPy, $\sigma = 0.2$, 50k steps	literal e^4 (9 seeds)	0.8–1.9 (λ_n at floor in 8 of 8)	4.7 modules, 0.3 hexagonal, 9% above 0.3
	tight e^{-4} (seed 0)	0.026 ($\lambda_n = 0.36$)	6 modules, 0 hexagonal, 22% above 0.3
JAX, $\sigma = 0.4$, $l = 0.5$, 150k steps, 3 seeds	tight 0.02	0.0209–0.0301 (held out)	median grid score 0.048–0.065; elongated periodic cells
	literal e^4	1.77–3.87 (held out)	median grid score -0.123 to -0.037; single large fields

Appendix: figures

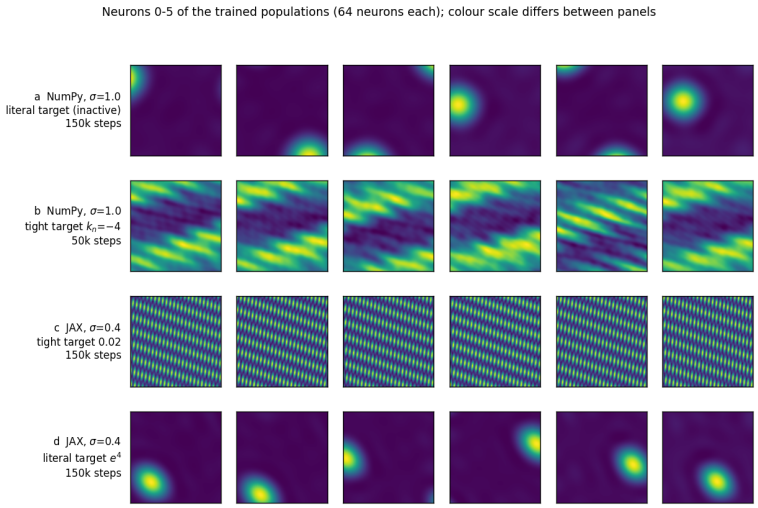


Figure A1: Rate maps of neurons 0–5 under a literal and a tight norm target, in both implementations (window $[-1.5, 1.5]^2$; colour scale differs between panels). Rows a and b are the NumPy runs at $\sigma = 1.0$; the module in row a is a hexagonal lattice of spacing 4.2 (from the autocorrelogram of the whole plane), larger than the window, and the module in row b has spacing 3.7 and is not hexagonal. Rows c and d are JAX seed-0 runs at $\sigma = 0.4$.

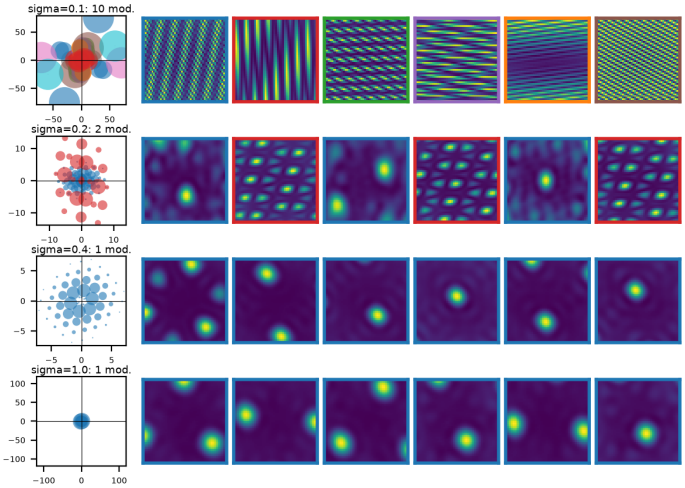


Figure A2: NumPy implementation, $l = 0.5$, 50k steps, seed 0: learned frequency power (left; one colour per module) and example rate maps (frame colour = module) at four values of σ. The module count falls from 10 at $\sigma = 0.1$ to 1 at $\sigma = 1.0$.

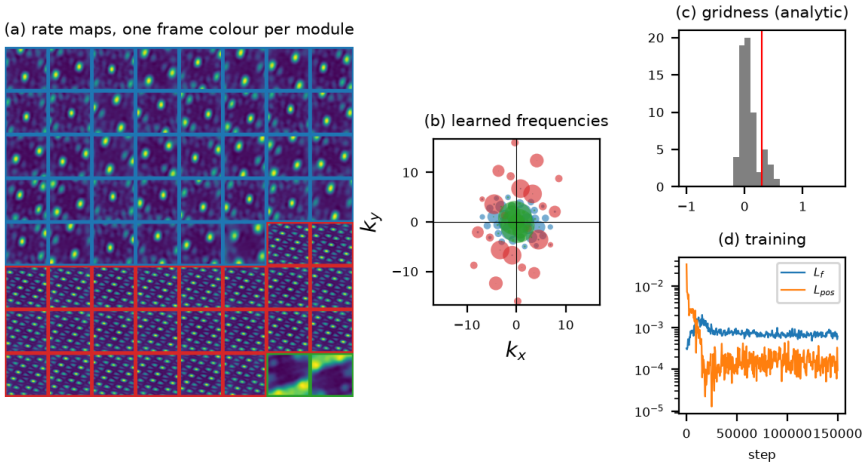


Figure A3: NumPy implementation at the settings of Figure 2B of the paper ($\sigma = 0.2$, $l = 0.5$, $N = 64$, 150k steps, seed 0): (a) rate maps, one frame colour per module, (b) learned frequencies, (c) histogram of gridness (red line: 0.3), (d) training losses. The three modules are not hexagonal.

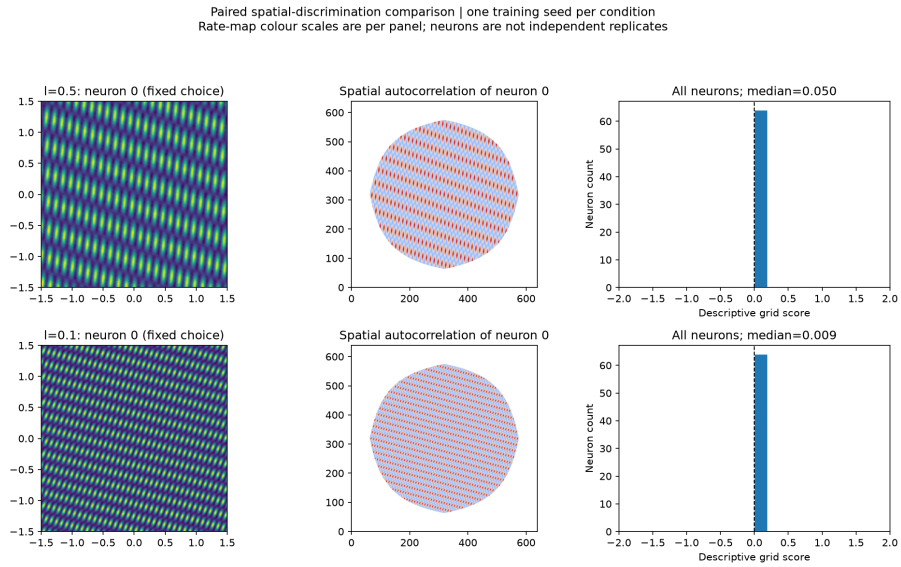


Figure A4: JAX implementation, the two seed-0 reference runs at $\sigma = 0.4$ (150k steps): neuron 0, its spatial autocorrelation, and the distribution of descriptive grid scores over all 64 neurons, for $l = 0.5$ (top) and $l = 0.1$ (bottom). Rate-map colour scales differ between panels; neurons are not independent replicates.

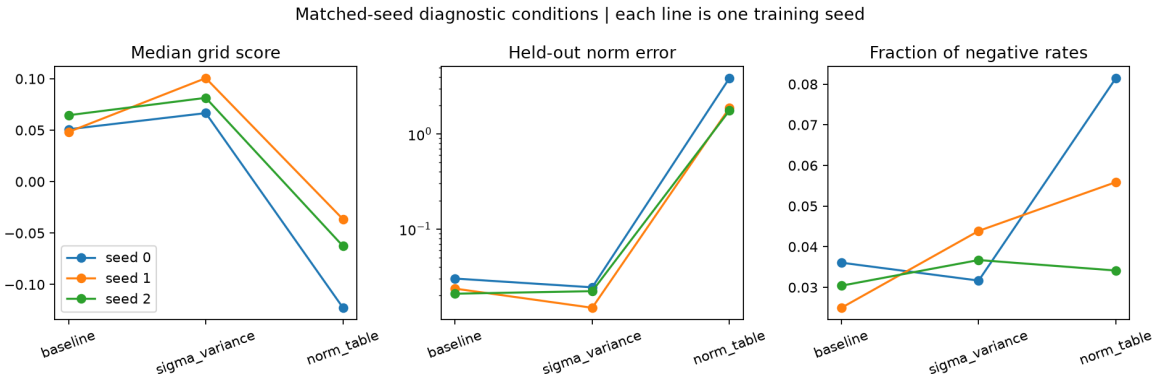


Figure A5: JAX implementation: all nine diagnostic checkpoints (150k steps each; lines connect matched seeds, initial states identical within a seed). Only the named setting changes: baseline uses $\sigma = 0.4$ and norm target 0.02; sigma_variance uses $\sigma = \sqrt{0.4}$; norm_table uses the literal norm target e^4. All evaluations use one common grid and held-out samples; negative rates are those below -10^{-6}; the norm axis is logarithmic.

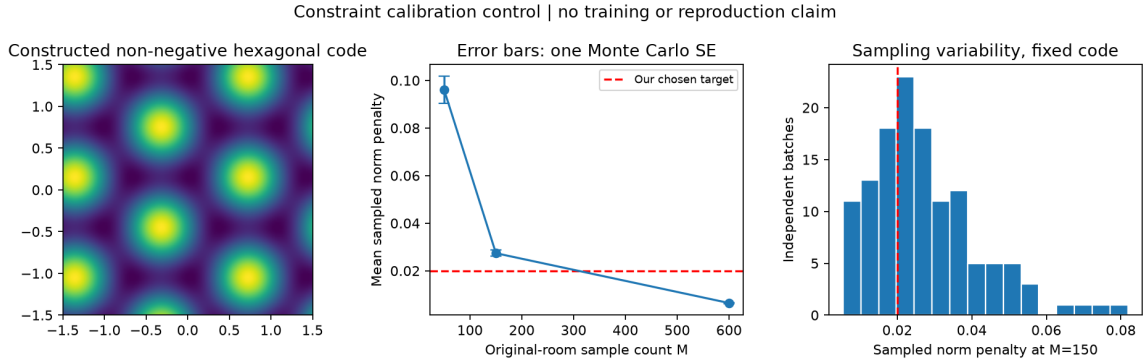


Figure A6: JAX implementation, constructed-code control (no training). Left: one neuron of a deliberately constructed non-negative hexagonal population. Centre: the sampled norm penalty falls as more positions are used to estimate room energy (error bars: one standard error over 128 sampling batches). Right: the distribution at 150 points; the 0.02 target is exceeded in most batches although the geometry is valid.

Appendix: Common evaluation of independent results

Table A3: Five Nathan checkpoints evaluated with Tafara's evaluator: eight held-out batches, seed 12345, sum normalisation and a 352×352 grid over $[-1.5, 1.5]^2$. The 0.02 norm tolerance is an evaluation choice; Nathan trained with e^4. The window under-resolves the repetition of broad grids, so negative scores do not reject their spectral geometry. These checkpoints are not independent comparisons of training implementations.

Checkpoint	Median grid score	Norm loss	Mean negativity
$\sigma = 0.2$, seed 0, 150k	0.0238	5.8261	0.0001878
$\sigma = 1.0$, seed 0, 150k	-0.0969	3.1925	0.0001943
$\sigma = 1.0$, seed 1, 150k	-0.1907	2.6302	0.0000858
$\sigma = 0.4$, seed 0, 50k	-0.0809	2.5480	0.0000770
$\sigma = 0.4$, seed 1, 50k	-0.0414	1.8502	0.0001619

The source checkpoints, conversion and evaluation script, checkpoint hashes, held-out summaries and plots accompany the submission. Training settings differ between the two implementations; agreement on the shared equation fixture does not imply identical optimisation trajectories. The inventory contains 60 completed folders but 59 distinct final parameter arrays, because two saved folders duplicate the same run.

NeurIPS checklist: reproduction submission

Answers apply to our reproduction code (both implementations), not to Dorrell et al.'s paper. The questions are abbreviated and paraphrased from the NeurIPS checklist.

1. **Are the claims supported and appropriately scoped? Yes.** The review separates the paper's claims, our reproduction and proposed follow-up work. We claim no successful reproduction of Figure 2B or Figure 8; Table A1 shows which claims held.
2. **Are limitations explained? Yes.** The Reproducibility section reports the seeds, step budgets, our own thresholds for "module" and "hexagonal", the tolerance choices, and what was not attempted.
3. **Are new theoretical claims proved? N/A.** We introduce no theorem; the reviewed theory is attributed to Dorrell et al.
4. **Can the numerical findings be checked? Yes.** Both implementations ship configurations, seeds, saved runs (NumPy: parameters and metrics of all 60 runs; JAX: checkpoints, metadata and evaluation summaries), unit tests, and a quick check, `smoke_test.py`. Every number in the review is a macro generated from these files.
5. **Are code, inputs, and instructions available? Yes.** The submitted archive contains all code, results, `README.txt` and `requirements.txt`. Positions are synthetic samples generated from recorded seeds; no external dataset is needed.
6. **Are experimental choices documented? Yes.** `code/numpy_impl/README.txt` and `code/jax_impl/references/implementation_notes.md` list the sampling scheme, budgets, optimiser, targets, and every choice the paper leaves open. A configuration accompanies each run.
7. **Is uncertainty quantified statistically? No (partly).** Most settings have one to three seeds. The 50k-step baseline has 9 seeds (mean $\pm$ SD) and the scaling slope has a bootstrap interval. Neurons are not treated as independent runs, and grid scores are descriptive, without shuffled significance tests.
8. **Are computing requirements fully reported? Partly.** NumPy: about 5.3 hours of summed recorded run durations for all 60 runs on a PC (a 150k-step run takes about 15 minutes). JAX: about 9 minutes per reference run (the two ran concurrently) and 71 minutes of training time summed over the nine diagnostic runs. CPU only. Peak memory and total preliminary compute were not measured.
9. **Does the research conform to the NeurIPS Code of Ethics? Yes.** We ran no human or animal experiments and used only simulated data.
10. **Are application-specific societal risks discussed? N/A.** This is a small synthetic modelling exercise with no deployed application or personal data. The relevant risk is scientific over-interpretation, addressed by explicit limits on our claims.
11. **Are special release safeguards required? N/A.** The released artefacts are a small spatial-representation model and synthetic numerical results.
12. **Are asset licences respected? Yes for third-party assets; our own code has no licence file.** NumPy, SciPy, Matplotlib and JAX are used through `pip` under their own licences, the course style file is used as supplied, and no code from the authors' repository is copied. A licence for our code would have to be added before any upstream contribution.
13. **Are new artefacts documented? Partly.** Methods, configurations, tests and limitations are documented in both implementations; redistribution licensing is unresolved.
14. **Are human-participant instructions and compensation described? N/A.** No participants or crowdworkers were recruited.
15. **Was ethics approval obtained where needed? N/A.** No human-subject research was performed and no animal data were collected.
16. **Is substantive LLM use disclosed? Yes.** The signed departmental declarations list the tools used and what each was used for: code generation, experiment automation and content summary. The package's `CONTRIBUTIONS.md` gives the same information in more detail.